

# SUKKUR IBA UNIVERSITY

Department of Computer Science

Web Engineering (CSE-311)

## Lab Report 04: CSS Layout

Flexbox, CSS Grid, Mobile-First Media Queries & Positioning

|                      |                                                                                                       |
|----------------------|-------------------------------------------------------------------------------------------------------|
| <b>Student Name:</b> | Muhammad Tayyab                                                                                       |
| <b>CMS ID:</b>       | 023-24-0118                                                                                           |
| <b>Course:</b>       | Web Engineering                                                                                       |
| <b>Semester:</b>     | 5th Semester (Fall 2026)                                                                              |
| <b>Submitted To:</b> | Lab Instructor                                                                                        |
| <b>Repository:</b>   | <a href="https://github.com/taybro-o/web-engineering-lab">github.com/taybro-o/web-engineering-lab</a> |

## 1. Warm-up: Flexbox vs Grid (warmup.html)

I opened warmup.html with Live Server to compare Flexbox and Grid with the same boxes.

- **Task 1 (Add box G):** In Flexbox (display: flex), all 7 boxes (A to G) stayed in one horizontal line and got thinner to fit. In Grid (grid-template-columns: 1fr 1fr 1fr), box G wrapped to a 3rd row right under box A.
- **Task 2 (flex-direction: column):** Adding flex-direction: column to .flex made the flex section stack vertically as one column. The grid section stayed exactly the same.

**Rule:** Flexbox is 1D (row or column), Grid is 2D (rows and columns).

File: Week 04/warmup.html

```
/* warmup.html */
.flex-row { display: flex; gap: 0.5rem; }
.flex-col { display: flex; flex-direction: column; gap: 0.5rem; }
.grid     { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 0.5rem; }
```

## 2. Class Activity A: Flexbox Gallery (gallery.html)

I created gallery.html with 6 boxes. On .gallery, I added display: flex and flex-wrap: wrap with gap: 1rem. On .photo, I added flex: 1 1 160px so the items start around 160px and wrap when screen width decreases.

When resizing the window in DevTools, the boxes wrapped from 4 to 2 to 1 per row. At 320px width there is no horizontal scrollbar.

File: Week 04/gallery.html (Activity A)

```
/* gallery.html - Activity A */
.gallery {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}
.photo {
  flex: 1 1 160px;
  background: #ddd;
  padding: 2rem 1rem;
  text-align: center;
}
```

## 3. Class Activity B: justify-content Testing

In gallery.html, I set .photo { flex: 0 0 120px; } so items have a fixed width and leave extra space on the main axis. Then I tested each justify-content value on .gallery:

- flex-start: Boxes pack at the left; extra space stays on the right.
- center: Boxes move to the center with equal space on left and right.
- flex-end: Boxes pack at the right edge; extra space stays on the left.
- space-between: First box is at the left edge, last box at the right edge, remaining space split evenly between boxes.
- space-around: Equal space on both sides of each box (outer space is half of inner space).
- space-evenly: Equal space between all boxes and edges.

#### 4. Class Activity C: Grid Gallery (grid-gallery.html)

Created grid-gallery.html with 6 photo boxes using CSS Grid instead of Flexbox.

Added display: grid; grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); gap: 1rem; on .gallery.

**Observation:** The layout reflows automatically as the browser width changes. It shows 4 columns on large screens, then 3, 2, and 1 column on small screens down to 320px with no horizontal scrollbar and no media queries needed.

File: Week 04/grid-gallery.html

```
/* grid-gallery.html - Activity C */
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 1rem;
}
```

#### 5. Class Activity D: Mobile-First Breakpoints (breakpoints.html)

In breakpoints.html, I removed auto-fit and wrote mobile-first media queries with min-width:

- **Base (no query, < 640px):** grid-template-columns: 1fr; (1 column for phones)
- **Small (@media (min-width: 640px)):** grid-template-columns: repeat(2, 1fr); (2 columns)
- **Medium (@media (min-width: 768px)):** grid-template-columns: repeat(3, 1fr); (3 columns)
- **Large (@media (min-width: 1024px)):** grid-template-columns: repeat(4, 1fr); (4 columns)
- **Extra large (@media (min-width: 1280px)):** grid-template-columns: repeat(4, 1fr); gap: 1.5rem; (4 columns + wider gap)

Tested in Chrome DevTools device toolbar at 320px, 640px, 768px, 1024px, and 1280px. Every breakpoint changed the columns as expected.

File: Week 04/breakpoints.html

```
/* breakpoints.html - Mobile-first media queries */
.gallery { display: grid; grid-template-columns: 1fr; gap: 1rem; }
@media (min-width: 640px) { .gallery { grid-template-columns: repeat(2, 1fr); } }
@media (min-width: 768px) { .gallery { grid-template-columns: repeat(3, 1fr); } }
@media (min-width: 1024px) { .gallery { grid-template-columns: repeat(4, 1fr); } }
@media (min-width: 1280px) { .gallery { grid-template-columns: repeat(4, 1fr); gap: 1.5rem; } }
```

#### 6. Class Activity E: CSS Positioning (position.html)

I tested all 5 CSS position values in position.html:

- **Static:** Normal flow. Boxes sit in HTML order with regular gaps and no offset.
- **Relative vs Absolute:** In relative, the middle box nudges by 22px but keeps its old seat (dotted ghost); other boxes do not move. In absolute, the mint box leaves the row completely and neighbours close the gap; it positions from the red dot (parent top-left with position: relative).
- **Fixed vs Sticky:** The fixed mint square stays pinned at bottom-right of the window the whole time while scrolling. The sticky mint square stays stuck at top: 8px while inside the pink container, and disappears when the pink container scrolls off screen.

##### Lab Questions:

1. Static is normal document flow in HTML order with no offset.
2. Relative keeps a gap; absolute leaves normal flow and lets neighbours close the gap.
3. Fixed follows the window; sticky follows its parent scrolling container.

File: Week 04/position.html

```
/* position.html */
.nudge { position: relative; top: 22px; left: 22px; }
.abs-frame { position: relative; min-height: 170px; }
.abs-child { position: absolute; top: 28px; left: 28px; }
.fixed-sq { position: fixed; right: 1rem; bottom: 1rem; z-index: 40; }
.sticky-sq { position: sticky; top: 8px; }
```